Name: Deras Pangestu Gusti (08)
Class: SIB 1G
NIM: 254107060114

JOBSHEET X
STACK

1. Learning Objectives

After completing this session, students will be able to::

1. Create a Stack data structure.
2. Implement Stack algorithms in a Java program.

2. Labs Activities

2.1 Experiment 1: Assignment Submission

Allocated time : 90 Minutes

A number of students submit their assignment files by stacking them on the lecturer's desk, applying the stack principle. The lecturer grades the assignments sequentially, starting from the top file. Refer to the following **Student** Class Diagram.

Student<AttendanceNumber>
nim: String name: String className: String grade: int
Student<AttendanceNumber>() Student<AttendanceNumber>(nim: String, name: String, className: String) grading(grade: int)

Next, to manage the submission of assignment files, a **StudentAssignmentStack** class is needed, serving as a Stack to store student assignment data. The attributes and methods within the **StudentAssignmentStack** class represent data processing using the Stack structure. Refer to the following **StudentAssignmentStack** Class Diagram.

StudentAssignmentStack <AttendanceNumber>
stack: Student[] size: int top: int
StudentAssignmentStack< AttendanceNumber >(size: int) isFull(): boolean isEmpty(): boolean push(mhs): void pop(): Student peek():Student print(): void

Note: *The data type of the stack variable should be adjusted according to the type of data to be stored in the Stack. In this exercise, the data to be stored is an array of* ***Student*** *objects; therefore, the data type used is* ***Student****.*

2.1.1 Experiment Steps

A. Student Class

1. Create a new folder named, for example **Jobsheet10** inside your repository. Then, create a new file and name it **Student<AttendanceNumber>**.java.
2. Complete the **Student** class by adding the attributes shown in the **Student** class diagram, which include **name**, **nim**, **className**, and **grade**.
3. Add a parameterized constructor to the **Student** class as specified in the **Student** class diagram. Set a default value of **gade = -1** to indicate that the assignment has not yet been graded.

```
Student(String nim, String name, String className){
        this.nim = nim;
        this.name = name;
        this.className = className;
        this.grade = -1;
    }
```

4. Add a **grading()** method to the **Student** class, which is used to set the grade when a student's assignment is evaluated.

```
void grading(int grade){
        this.grade = grade;
    }
```

B. StudentAssignmentStack Class

5. After creating the **Student** class, you will then create the **StudentAssignmentStack** <AttendanceNumber>.java class to manage the stack of assignments. The **StudentAssignmentStack** class is an implementation of the Stack data structure.
6. Complete the **StudentAssignmentStack** class by adding the attributes shown in the **StudentAssignmentStack** class diagram, which include **stack**, **size**, and **top**.

```
Student[] stack;
int top, size;
```

7. Add a parameterized constructor to the **StudentAssignmentStack** class to initialize the maximum capacity of student assignment data that can be stored in the Stack, as well as to set the initial index of the **top** pointer.

```
StudentAssignmentStack(int size){
        this.size = size;
        top = -1;
        stack = new Student[size];
    }
```

8. Next, create a boolean **isFull** method to check whether the stack of student assignments has reached its maximum capacity.

```
boolean isFull(){
        if(top==size-1){
            return true;
        }else{
            return false;
        }
    }
```

9. Then, create another boolean **isEmpty** method to check whether the stack of assignments is empty.

```
boolean isEmpty(){
        if(top==-1){
            return true;
        }else{
            return false;
        }
    }
```

10. To add assignment files to the Stack, create a **push** method. This method should accept a parameter **std**, which is an object of the **Student** class.

```
void push(Student std){
        if(!isFull()){
            top++;
            stack[top] = std;
        }else{
            System.out.println("Stack is already full!!");
        }
    }
```

11. The grading of student assignments by the lecturer is done using the **pop** method to remove the assignment to be graded. This method does not accept any parameters but returns an object of the **Student** class.

```
Student pop(){
        if(!isEmpty()){
            Student std = stack[top];
            top--;
            return std;
        }else{
            System.out.println("There is no data in Stack!!");
            return null;
        }
    }
```

***Note**: If information about the student data is required, the return type should be an object of the **Student** class. Conversely, a **void** return type can be used if the student data that is removed will not be processed or used again.*

12. Create a **peek** method to check the student assignment at the **top** of the stack.

```
Student peek(){
        if(!isEmpty()){
            return stack[top];
        }else{
            System.out.println("There is no data in Stack!!");
            return null;
        }
    }
```

13. Add a **print** method to display the entire list of student assignments in the Stack.

```
void print(){
        for(int i=0;i<=top;i++){
            System.out.println(stack[i].nim+"\t"+stack[i].name+"\t"
            +stack[i].className);
        }
        System.out.println("");
    }
```

C. Main Class

14. Create a new main class named **StudentDemo<AttendanceNumber>.java**
15. Add a new **main()** method
16. Within main() method, instantiate a new object from **StudentAssignmentStack** class. Give the object name as **stack** and pass **5** as the parameter.

```
StudentAssignmentStack stack = new StudentAssignmentStack(5);
```

17. Declare a Scanner object with the variable name **scan** and an int variable named **choice**.

18. Add a menu to allow users to choose Stack operations for managing student assignment data, using a **do-while** loop structure.

```java
do {
    System.out.println("\nMenu:");
    System.out.println("1. Mengumpulkan Tugas");
    System.out.println("2. Menilai Tugas");
    System.out.println("3. Melihat Tugas Teratas");
    System.out.println("4. Melihat Daftar Tugas");
    System.out.print("Pilih: ");
    pilih = scan.nextInt();
    scan.nextLine();
    switch (pilih) {
        case 1:
            System.out.print("Nama: ");
            String nama = scan.nextLine();
            System.out.print("NIM: ");
            String nim = scan.nextLine();
            System.out.print("Kelas: ");
            String kelas = scan.nextLine();
            Mahasiswa mhs = new Mahasiswa(nama, nim, kelas);
            stack.push(mhs);
            System.out.printf("Tugas %s berhasil dikumpulkan\n", mhs.nama);
            break;
        case 2:
            Mahasiswa dinilai = stack.pop();
            if (dinilai != null) {
                System.out.println("Menilai tugas dari " + dinilai.nama);
                System.out.print(s:"Masukkan nilai (0-100): ");
                int nilai = scan.nextInt();
                dinilai.tugasDinilai(nilai);
                System.out.printf(format:"Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
            }
            break;
        case 3:
            Mahasiswa lihat = stack.peek();
            if (lihat != null) {
                System.out.println("Tugas terakhir dikumpulkan oleh " + lihat.nama);
            }
            break;
        case 4:
            System.out.println(x:"Daftar semua tugas");
            System.out.println(x:"Nama\tNIM\tKelas");
            stack.print();
            break;
        default:
            System.out.println(x:"Pilihan tidak valid.");
    }
} while (pilih >= 1 && pilih <= 4);
```

19. **Commit and push your code to your Github repository**

20. Compile and run your program.

```java
package com.jobsheet10;

public class Student8
{
    String  name,
            className,
            nim;
    int     grade;

    Student8(String nim, String name, String className)
    {
        this.nim = nim;
        this.name = name;
        this.className = className;
        this.grade = -1;
    }

    void grading (int grade)
    {
        this.grade = grade;
    }
}
```

```java
package com.jobsheet10;

public class StudentAssigmentStack8
{
    Student8[] stack;
    int top,
        size;

    public StudentAssigmentStack8 (int size)
    {
        this.size = size;
        top = -1;
        stack = new Student8[size];
    }

    boolean isFull ()
    {
        if (top == size - 1)
            return true;
        else
            return false;
    }

    boolean isEmpty ()
    {
        if (top == -1)
            return true;
        else
            return false;
    }

    void push (Student8 std)
    {
        if (!isFull())
        {
            top++;
            stack[top] = std;
        }
        else
            System.out.println(x: "Stack is already full!!");
    }

```

```java
    Student8 pop ()
    {
        if (!isEmpty())
        {
            Student8 std = stack[top];
            top--;
            return std;
        }
        else
        {
            System.out.println(x: "There's no data in Stack!!");
            return null;
        }
    }

    Student8 peek ()
    {
        if (!isEmpty())
            return stack[top];
        else
        {
            System.out.println(x: "There's no data in Stack!!");
            return null;
        }
    }

    void print()
    {
        for (int i = 0; i <= top; i++)
        {
            System.out.println
            (
                stack[i].nim + "\t" +
                stack[i].name + "\t" +
                stack[i].className
            );
        }
        System.out.println(x: "");
    }
}
```

```java
package com.jobsheet10;
import java.util.Scanner;
public class StudentDemo8
{
    Run | Debug | Run main | Debug main
    public static void main(String[] args)
    {
        Scanner scan = new Scanner(System.in);

        StudentAssigmentStack8 stack = new StudentAssigmentStack8(size: 5);

        int select;

        do
        {
            System.out.println
            (
                "\nMenu: \n" +
                "1. Submitting Assignments\n" +
                "2. Assessing Assignments\n" +
                "3. View Top Assigment\n" +
                "4. View the To-Do List" +
                "0. Exit"
            );

            System.out.print(s: "Select: ");
            select = scan.nextInt();
            scan.nextLine();

            switch (select)
            {
                case 1:
                    System.out.print(s: "\nName: ");
                    String name = scan.nextLine();
                    System.out.print(s: "NIM: ");
                    String nim = scan.nextLine();
                    System.out.print(s: "Class: ");
                    String studentClass = scan.nextLine();

                    Student8 std = new Student8(nim, name, studentClass);
                    stack.push(std);

                    System.out.printf(format: "%s's assignment has been successfully submitted\n", std.name);
                    break;

```

```java
                case 2:
                    Student8 assessed = stack.pop();
                    if (assessed != null)
                    {
                        System.out.printf(format: "\nViewing %s assignment\n", assessed.name);
                        System.out.print(s: "Input the value (0-100): ");
                        int value = scan.nextInt();
                        assessed.grading(value);
                        System.out.printf(format: "%s mark for the assignment is %d\n", assessed.name, value);
                    }
                    break;

                case 3:
                    Student8 see =  stack.peek();
                    if (see != null)
                        System.out.println("The final assignment was handed in by " + see.name);
                    break;

                case 4:
                    System.out.println
                    (
                        "\nList of Assigments\n" +
                        "Name\tNIM\tClass"
                    );
                    stack.print();
                    break;

                case 0:
                    System.out.println(x: "Thankyou :D");
                    return;

                default:
                    System.out.println(x: "Invalid selection");
                    continue;
            }
        } while (select >= 1 && select <= 4);

        scan.close();
    }
}
```

```
Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 1

Name: Bima
NIM: 1001
Class: 1I
Bima's assignment has been successfully submitted

Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 1

Name: Fidela
NIM: 1002
Class: 1I
Fidela's assignment has been successfully submitted

Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 1

Name: Resty
NIM: 1003
Class: 1I
Resty's assignment has been successfully submitted

Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 3
The final assignment was handed in by Resty
```

```
Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 1

Name: Joseph
NIM: 1004
Class: 1I
Joseph's assignment has been successfully submitted

Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 4

List of Assigments
Name    NIM     Class
1001    Bima    1I
1002    Fidela  1I
1003    Resty   1I
1004    Joseph  1I


Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 2

Viewing Joseph assignment
Input the value (0-100): 85
Joseph mark for the assignment is 85
```

```
Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 4

List of Assigments
Name    NIM     Class
1001    Bima    1I
1002    Fidela  1I
1003    Resty   1I


Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
0. Exit
Select: 0
Thankyou :D
```

2.1.2 Verification

Compare the output of your compiled program with the following output.

```
Menu:
```

```
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Bima
NIM: 1001
Class Name: 1I
Bima's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Fidela
NIM: 1002
Class Name: 1I
Fidela's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Resty
NIM: 1003
Class Name: 1I
Resty's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 3
The last assignmentcomes from Resty

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Joseph
NIM: 1004
Class Name: 1I
Joseph's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 4
Assignment list:
Name     NIM      Class Name
1001     Bima     1I
1002     Fidela   1I
1003     Resty    1I
1004     Joseph   1I

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
```

```
4. View All Assignments
Choose a menu: 2
Grading assignment from Joseph
Input grade (0-100): 85
Assignment grade of Joseph is 85

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 4
Assignment list:
Name     NIM      Class Name
1001     Bima     1I
1002     Fidela   1I
1003     Resty    1I

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu:
```

2.1.3 Questions

1. Explain the role of the stack data structure in the student assignment management system. Why was a stack used instead of another data structure (e.g., queue or list)?

   The stack implements a Last In First Out mechanism so the most recently submitted assignment is assessed first. This matches a workflow where assessors pick the latest submission to mark immediately. A stack was chosen because the program needs LIFO behaviour rather than FIFO behaviour that a queue would provide. A simple list could store items but would require extra logic to enforce LIFO consistently.

2. What is the difference between the **push()** and **pop()** methods in a stack, and how are they used in this program?

   push() adds a Student8 object to the top of the stack. pop() removes and returns the object currently at the top. In this program push() is used when a student submits an assignment so their Student8 instance is placed on top. pop() is used when an assessor takes the next assignment to mark, removing that most recent submission from the stack.

3. Why is it important to check the condition **!isFull()** before calling the **push()** method? What could go wrong if this check is removed?

   Checking !isFull() prevents adding an element when the underlying array is already at capacity. Without that check the code would increment top beyond the array bounds and cause an ArrayIndexOutOfBoundsException or corrupt memory. The check ensures safe insertion and preserves program stability.

4. How many student assignments can be stored in the current implementation of the stack? Provide the specific line of code or variable that determines this.

```
this.size = size;
top = -1;
stack = new Student8[size];
```

*In the demo the stack is created with size 5 using new StudentAssignmentStack8(5) so the current implementation holds up to **5** assignments.*

5. Modify the existing stack implementation so that the system can also show **the first student who submitted their assignment**. Describe the changes you made in both **StudentDemo** and **StudentAssignmentStack** classes.

```
    Student8 firstSubmission()
    {
        if (!isEmpty())
            return stack[0];
        else
            return null;
    }
```

```
            case 5:
                Student8 first = stack.firstSubmission();
                if (first != null)
                    System.out.println("First submission by " + first.name + " NIM: " + first.nim);
                else
                    System.out.println(x: "No submissions yet");
                break;

```

Add a method that returns the bottom element of the stack. The bottom element is at index 0 while top indicates the current top. If the stack is empty return null. Update the demo to call this method and print the result.

6. Implement a method to count and return the number of assignments currently stored in the stack. Describe how your method works.

```
    int count()
    {
        return top + 1;
    }
```

When the stack is empty top is -1 so top + 1 yields 0. Each push() increments top and each pop() decrements top, so top + 1 always reflects the current element count.

7. What did you learn from this experiment about stack-based systems? Reflect on a real-world application where this kind of system might be useful.

   *This experiment highlights the simplicity and predictability of LIFO structures. Stacks are excellent when the most recent item must be handled first. They are memory efficient and easy to implement with an array and a single index. A real-world application is an **undo** system in text editors where the last action is reverted first. Other examples include browser history navigation, expression evaluation in compilers, and call stacks in program execution.*

8. Don't forget to synchronize the changes you made from this questions into your repository!

```
PS C:\Users\deras\OneDrive\Documents\POLINEMA\Semester 2\Praktikum Algoritma & Struktur Data> git add .
PS C:\Users\deras\OneDrive\Documents\POLINEMA\Semester 2\Praktikum Algoritma & Struktur Data> git commit -m "Update Experiment 1"
[main a999baf] Update Experiment 1
 20 files changed, 233 insertions(+), 19 deletions(-)
 create mode 100644 bin/com/cm1/Book.class
 create mode 100644 bin/com/cm1/BookLending.class
 create mode 100644 bin/com/cm1/Main.class
 create mode 100644 bin/com/cm1/Student.class
 create mode 100644 bin/com/jobsheet10/Student8.class
 create mode 100644 bin/com/jobsheet10/StudentAssigmentStack8.class
 create mode 100644 bin/com/jobsheet10/StudentDemo8.class
 delete mode 100644 bin/com/jobsheet6/StydentDemo8.class
 create mode 100644 bin/com/test/Main.class
 create mode 100644 file/jobsheet10/DerasPangestuGusti_Jobsheet10_PASD.docx
 create mode 100644 file/jobsheet10/Jobsheet 10 - Stack - en.pdf
 create mode 100644 src/com/jobsheet10/Student8.java
 create mode 100644 src/com/jobsheet10/StudentAssigmentStack8.java
 create mode 100644 src/com/jobsheet10/StudentDemo8.java
 create mode 100644 src/com/test/Main.java
PS C:\Users\deras\OneDrive\Documents\POLINEMA\Semester 2\Praktikum Algoritma & Struktur Data> git push origin main
Enumerating objects: 49, done.
Counting objects: 100% (49/49), done.
Delta compression using up to 20 threads
Compressing objects: 100% (30/30), done.
Writing objects: 100% (35/35), 1.43 MiB | 576.00 KiB/s, done.
Total 35 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), completed with 8 local objects.
To https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data.git
   472c320..a999baf  main -> main
```

https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet10

2.2 Experiment 2: Convert Assignment Grade to Binary

Time Allocated: 60 Minutes

In Experiment 2, a new method is added that functions to convert the task value of type **int** into **binary** form after the task has been assigned a value and removed from the Stack.

2.2.1 Experiment Steps

1. Re-open **StudentAsignmentStack<AttendanceNumber>.java**
2. Create a new method named **convertToBinary()** with an **int** parameter **grade**

```
String convertToBinary(int grade){
        ConversionStack stack = new ConversionStack();
        while (grade > 0) {
            int mod = grade % 2;
            stack.push(mod);
            grade = grade / 2;
        }
        String binary = "";
        while (!stack.isEmpty()) {
            binary += stack.pop();
        }
        return binary;
    }
```

In this method, the use of **ConversionStack** is implemented, which is similar to the **StudentAssignmentStack** class. The purpose of this is to differentiate the Stack used for students from the Stack used for binary data, as the data types involved are different. Therefore, create a new file named **ConversionStack<AttendanceNumber>.java**.

Note: *It is important to remember that all Stack classes essentially have the same operations (methods). What distinguishes them is the specific activities that need to be performed, such as after adding or removing data.*

```
public class ConversionStack {
    int[] binaryStack;
    int size;
    int top;

    public ConversionStack() {
        this.size = 32; //asumsi 32 bit
        binaryStack = new int[size];
        top = -1;
    }
    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == size - 1;
    }
```

```
    public void push(int data) {
        if (isFull()) {
            System.out.println("Stack is already full-filled!!");
        } else {
            top++;
            binaryStack[top] = data;
        }
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack is still empty!!");
            return -1;
        } else {
            int data = binaryStack[top];
            top--;
            return data;
        }
    }
}
```

3. To convert the student's assignment value into **binary** form after grading, add a line of code in StudentDemo class, specifically within case 2.

```
case 2:
      Student graded = stack.pop();
     if (graded != null) {
       System.out.println("Grading assignment from " + graded.name);
       System.out.print("Input grade (0-100): ");
       int grade = scan.nextInt();
       graded.grading(grade);
       System.out.printf("Assignment grade of %s is %d\n", graded.name,
grade);
       String binary = stack.convertToBinary(grade);
       System.out.printf("Assignment grade in binary is %s\n", binary);
     }
     break;
```

4. Compile and run the program.

```java
        String convertToBinary (int grade)
        {
            ConversionStack stack = new ConversionStack();
            while (grade > 0)
            {
                int mod = grade % 2;
                stack.push(mod);
                grade = grade / 2;
            }

            String binary = "";
            while (!stack.isEmpty())
                binary += stack.pop();

            return binary;
        }
    }
```

```java
package com.jobsheet10;

public class ConversionStack
{
    int[] binaryStack;
    int size,
        top;

    public ConversionStack()
    {
        this.size = 32;
        binaryStack = new int[size];
        top = -1;
    }

    public boolean isEmpty()
    {
        return top == -1;
    }

    public boolean isFull()
    {
        return top == size - 1;
    }

    public void push (int data)
    {
        if (isFull())
            System.out.println(x: "Stack is already full-filled!!");
        else
        {
            top++;
            binaryStack[top] = data;
        }
    }

    public int pop()
    {
        if (isEmpty())
            {
                System.out.println(x: "Stack is still empty!!");
                return -1;
            }
        else
        {
            int data = binaryStack[top];
            top--;
            return data;
        }
    }
}
```

```java
        case 2:
            Student8 assessed = stack.pop();
            if (assessed != null)
            {
                System.out.printf(format: "\nViewing %s assignment\n", assessed.name);
                System.out.print(s: "Input the value (0-100): ");
                int value = scan.nextInt();
                assessed.grading(value);
                System.out.printf(format: "%s mark for the assignment is %d\n", assessed.name, value);

                String binary = stack.convertToBinary(value);
                System.out.printf(format: "Assigment grade in binary is %s\n", binary);
            }
            break;

```

```
Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
5. View First Submission
0. Exit
Select: 1

Name: Excel
NIM: 1001
Class: 1I
Excel's assignment has been successfully submitted

Menu:
1. Submitting Assignments
2. Assessing Assignments
3. View Top Assigment
4. View the To-Do List
5. View First Submission
0. Exit
Select: 2

Viewing Excel assignment
Input the value (0-100): 87
Excel mark for the assignment is 87
Assigment grade in binary is 1010111
```

5. **Commit and push the code to your Github repository.**

```
PS C:\Users\deras\OneDrive\Documents\POLINEMA\Semester 2\Praktikum Algoritma & Struktur Data> git add .
PS C:\Users\deras\OneDrive\Documents\POLINEMA\Semester 2\Praktikum Algoritma & Struktur Data> git commit -m "Update Experiment 2"
[main ffe69ab] Update Experiment 2
 7 files changed, 95 insertions(+), 2 deletions(-)
 create mode 100644 bin/com/jobsheet10/ConversionStack.class
 create mode 100644 src/com/jobsheet10/ConversionStack.java
PS C:\Users\deras\OneDrive\Documents\POLINEMA\Semester 2\Praktikum Algoritma & Struktur Data> git push origin main
Enumerating objects: 30, done.
Counting objects: 100% (30/30), done.
Delta compression using up to 20 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (17/17), 409.33 KiB | 15.16 MiB/s, done.
Total 17 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), completed with 8 local objects.
To https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data.git
   a999baf..ffe69ab  main -> main
```

https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet10

2.2.2 Verification

Compare the output of your compiled program with the following output.

```
Menu:
1. Submit Assignment
2. Grade Assignment
```

```
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Excel
NIM: 1001
Class Name: 1I
Excel's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 2
Grading assignment from Excel
Input grade (0-100): 87
Assignment grade of Excel is 87
Assignment grade in binary is 1010111
```

2.2.3 Questions

1. Explain the workflow of the **convertToBinary()** method.

   a. *First, a ConversionStack object is created to store remainders.*

   b. *The method repeatedly divides the integer by 2, pushing the remainder (grade % 2) onto the stack. This ensures that the least significant bit is stored first.*

   c. *Once the integer becomes 0, the loop ends.*

   d. *Then, the method pops all elements from the stack one by one, appending them to a string. Because stacks are LIFO, this reverses the order of remainders, producing the correct binary sequence.*

   e. *Finally, the binary string is returned.*

2. In the **convertToBinary()** method, change the loop condition to **while (grade != 0)**. What is the result? Explain the reason!

```java
    String convertToBinary (int grade)
    {
        ConversionStack stack = new ConversionStack();
        while (grade != 0)
        {
            int mod = grade % 2;
            stack.push(mod);
            grade = grade / 2;
        }

        String binary = "";
        while (!stack.isEmpty())
            binary += stack.pop();

        return binary;
    }
}
```

The result is essentially the same as while (grade > 0) because both conditions terminate when grade reaches 0.

With while (grade > 0), the loop only runs when the integer is positive. If grade is 0 at the start, the loop is skipped entirely, and the binary string remains empty. With while (grade != 0), the loop would also run for negative values, which is not intended in this context. Negative integers would cause an infinite loop because dividing by 2 never reaches exactly 0.

2.4 Assignment

Allocated Time : 90 Minutes

Students submit an excuse letter (due to illness or other reasons) each time they are absent from a lecture. The most recent letter submitted will be processed or validated first by the department administrator. Refer to the class diagram below.

ExcuseLetter<AttendanceNumber>
id: String name: String className: String typeOfExcuse: char duration: int
ExcuseLetter<AttendanceNumber>() ExcuseLetter<AttendanceNumber>(id: String, name: String, className: String, type: char, duration: int)

The **typeOfExcuse** attribute is used to store the type of student leave (S: Sick, or I: Other personal reasons), while the **duration** attribute stores the length of the leave period.

Based on the class diagram, implement a class named **ExcuseLetter**, and add a **ExcuseLetterStack** class to manage the collection of **ExcuseLetter** objects. In the class containing the **main** method, provide the following menu options:

1. **Submit Excuse Letter** – to input new excuse letter data
2. **Process Excuse Letter** – to process or verify the latest excuse letter

3. **View Latest Excuse Letter** – to display the top letter in the stack
4. **Search for Letter** – to search for the presence of a letter by the **student's name**

```java
package com.jobsheet10;

public class ExcuseLetter8
{
    String  id,
            name,
            className;
    char    typeOfExcuse;
    int     duration;

    public ExcuseLetter8(String id, String name, String className, char type, int duration)
    {
        this.id = id;
        this.name = name;
        this.className = className;
        typeOfExcuse = type;
        this.duration = duration;
    }
}
```

```java
package com.jobsheet10;

public class ExcuseLetterStack8
{
    ExcuseLetter8[] stack;
    int top,
        size;

    public ExcuseLetterStack8(int size)
    {
        this.size = size;
        top = -1;
        stack = new ExcuseLetter8[size];
    }

    boolean isFull ()
    {
        if (top == size - 1)
            return true;
        else
            return false;
    }

    boolean isEmpty ()
    {
        if (top == -1)
            return true;
        else
            return false;
    }

    void push (ExcuseLetter8 ltr)
    {
        if (!isFull())
        {
            top++;
            stack[top] = ltr;
        }
        else
            System.out.println(x: "Stack is already full!!");
    }

```

```java
    ExcuseLetter8 pop ()
    {
        if (!isEmpty())
        {
            ExcuseLetter8 ltr = stack[top];
            top--;
            return ltr;
        }
        else
        {
            System.out.println(x: "There's no data in Stack!!");
            return null;
        }
    }

    ExcuseLetter8 peek ()
    {
        if (!isEmpty())
            return stack[top];
        else
        {
            System.out.println(x: "There's no data in Stack!!");
            return null;
        }
    }

```

```java
    void Searching (String search)
    {
        boolean key = true;
        for (int i = 0; i < size; i++)
        {
            if (stack[i] != null && stack[i].name.equals(search))
            {
                System.out.printf
                (
                    """
                    The letter has been found!
                    Name        : %s
                    Reason      : %c
                    Duration    : %d
                    """
                    , stack[i].name, stack[i].typeOfExcuse, stack[i].duration
                );
                key = false;
                break;
            }
        }

        if (key != false)
        {
            System.out.println(x: "The letter was not found!");
        }
    }
}
```

```java
package com.jobsheet10;

import java.util.Scanner;

public class ExcuseLetterMain8
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        ExcuseLetterStack8 stack = new ExcuseLetterStack8(size: 5);

        do
        {
            System.out.print
            (
                """
                \nMenu:
                1. Submit Excuse Letter
                2. Process Excuse Letter
                3. View Latest Excuse Letter
                4. Search for Letter
                0. Exit
                """
            );
            System.out.print(s: "Select: ");
            int select = sc.nextInt();
            sc.nextLine();

```

```java
            switch (select)
            {
                case 1:
                    System.out.print(s: "\nID: ");
                    String id = sc.nextLine();
                    System.out.print(s: "Name: ");
                    String name = sc.nextLine();
                    System.out.print(s: "Class: ");
                    String className = sc.nextLine();
                    System.out.print
                    (
                        """
                        Reason Leave:
                        s: Sick
                        i: Other personal reason
                        """
                    );
                    System.out.print(s: "Select: ");
                    char type = sc.nextLine().charAt(index: 0);
                    System.out.print(s: "Duration (days): ");
                    int duration = sc.nextInt();
                    sc.nextLine();

                    ExcuseLetter8 ltr = new ExcuseLetter8(id, name, className, type, duration);
                    stack.push(ltr);

                    System.out.printf(format: "%s's Excuse Letter has been successfully submitted\n", ltr.name);
                    break;

```

```java
                case 2:
                    ExcuseLetter8 letter = stack.pop();
                    if (letter != null)
                    {
                        System.out.printf
                        (
                            """
                            Processing the excuse letter...
                            Name        : %s
                            Reason      : %c
                            Duration    : %d days
                            The excuse later has been verified.\n
                            """
                            , letter.name, letter.typeOfExcuse, letter.duration
                        );
                    }
                    break;

                case 3:
                    ExcuseLetter8 top = stack.peek();
                    if (top != null)
                        System.out.println("The latest excuse letter is by " + top.name + "'s letter");
                    break;

                case 4:
                    System.out.println(x: "Search for student's letter");
                    System.out.print(s: "Input the name: ");
                    String studentName = sc.nextLine();
                    stack.Searching(studentName);
                    break;

                case 0:
                    System.out.println(x: "Good bye.");
                    sc.close();
                    return;

                default:
                    System.out.println(x: "Input Invalid!! Please, Try Again!");
            }
        }
        while (true);
    }
}
```

```
Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 2
There's no data in Stack!!

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 3
There's no data in Stack!!

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 4
Search for student's letter
Input the name: Deras
The letter was not found!

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 5
Input Invalid!! Please, Try Again!
```

```
Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 1

ID: 0001
Name: Deras
Class: A
Reason Leave:
s: Sick
i: Other personal reason
Select: s
Duration (days): 3
Deras's Excuse Letter has been successfully submitted

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 1

ID: 0002
Name: Gurami
Class: A
Reason Leave:
s: Sick
i: Other personal reason
Select: i
Duration (days): 5
Gurami's Excuse Letter has been successfully submitted
```

```
Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 1

ID: 0003
Name: Dadar
Class: A
Reason Leave:
s: Sick
i: Other personal reason
Select: i
Duration (days): 7
Dadar's Excuse Letter has been successfully submitted

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 3
The latest excuse letter is by Dadar's letter

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 2
    Processing the excuse letter...
    Name        : Dadar
    Reason      : i
    Duration    : 7 days
    The excuse later has been verified.
```

```
Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 3
The latest excuse letter is by Gurami's letter

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 2
    Processing the excuse letter...
    Name        : Gurami
    Reason      : i
    Duration    : 5 days
    The excuse later has been verified.


Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 3
The latest excuse letter is by Deras's letter

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 2
    Processing the excuse letter...
    Name        : Deras
    Reason      : s
    Duration    : 3 days
    The excuse later has been verified.
```

```
Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 4
Search for student's letter
Input the name: Deras
    The letter has been found!
    Name        : Deras
    Reason      : s
    Duration    : 3

Menu:
1. Submit Excuse Letter
2. Process Excuse Letter
3. View Latest Excuse Letter
4. Search for Letter
0. Exit
Select: 0
Good bye.
```

Github:

https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet10